Graph Structure Learning for Traffic Prediction

Mahmood Amintoosi[1*]

[1*]Division of Computer Science, Department of Applied Mathematics, Faculty of Mathematical Sciences, Ferdowsi University of Mashhad, Bahonar Av., Mashhad, 91775, Khorasan-e Razavi, IRAN, ORCID:0000-0001-9640-6475.

Corresponding author(s). E-mail(s): m.amintoosi@um.ac.ir;

Abstract

Graph-based traffic forecasting methods typically rely on predefined adjacency matrices derived from physical road proximity. However, dense physical connectivity can induce excessive spatial aggregation—a form of oversmoothing—that degrades prediction accuracy. We propose a multi-lag graph structure learning framework that discovers temporally heterogeneous functional dependencies between traffic sensors. Using DAGMA continuous optimization, we construct lag-specific dependency graphs from explicit temporal input blockings of the form $\boldsymbol{Z} = [\boldsymbol{x}(\boldsymbol{t}-\boldsymbol{L}), \ldots, \boldsymbol{x}(\boldsymbol{t})]$, yielding separate functional graphs for each temporal lag. A T-GCN-MultiGSL-Mix architecture then combines these lag-specific graphs through a per-node, per-timestep learned mixing gate. Experiments on the Los-loop highway dataset demonstrate a mean RMSE improvement of 14.9% over a no-graph baseline across five random seeds at 5-minute sampling, increasing to 27.4% at 15-minute sampling, with the improvement robust to a parameter-matched control and to matched-edge-budget sparse-graph controls. The original submission's central result—a sparse learned DAG substantially outperforming the dense physical graph—is reproduced under the revised protocol, with a mean improvement of 21.7% across five seeds and the historically submitted graphs recovered at the support level. A lag ablation study confirms that all three temporal lags contribute complementary information. On the SZ-Taxi urban dataset, improvements are marginal, indicating dataset dependence. These findings establish that temporally heterogeneous graph structures can substantially benefit traffic forecasting when the underlying dependency patterns vary across temporal scales.

Keywords: Traffic Forecasting, Graph Structure Learning, Multi-Lag Dependency Learning, Graph Convolutional Networks, Learned Graph Mixing

1 Introduction

Traffic prediction is a fundamental challenge in intelligent transportation systems, requiring models that capture complex spatial and temporal dependencies in urban road networks [1]. Graph Convolutional Networks (GCNs) [2] and Temporal GCNs (T-GCNs) [3] have emerged as effective approaches by modeling traffic networks as graphs, where nodes represent road segments and edges encode spatial relationships.

A critical limitation of existing graph-based methods is their reliance on predefined adjacency matrices derived from physical road proximity [3]. A systematic bibliometric analysis of 784 traffic forecasting articles (2000–2025) confirms that graph-based methods now dominate the field, yet nearly all depend on heuristic graph construction from road network topology (see Appendix B). However, physical connectivity is an imperfect proxy for functional traffic dependency: roads that are physically distant may exhibit strong traffic correlations due to commuting patterns, while adjacent roads may have minimal mutual influence [4].

A second, less recognized problem is that dense physical graphs can induce *over-smoothing* in GCN-based models. When the adjacency matrix encodes hundreds or thousands of edges, repeated graph convolution operations aggregate information from increasingly distant nodes, causing node representations to converge and lose discriminative power. This excessive spatial aggregation can paradoxically degrade forecasting performance compared to using no graph at all.

Graph Structure Learning (GSL) addresses the first problem by learning the adjacency matrix directly from traffic data rather than relying on physical proximity. Recent work has applied continuous DAG optimization methods such as NOTEARS [5] and DAGMA [6] to this task, discovering sparse functional dependencies that differ substantially from physical road connectivity [7].

However, a single static learned graph has an inherent limitation: traffic dependencies are *temporally heterogeneous*. The influence of sensor i on sensor j may vary depending on the temporal lag—a 5-minute delay captures different traffic propagation patterns than a 15-minute delay. A single adjacency matrix cannot represent these lag-specific structures.

In this paper, we address this limitation through two contributions:

1. **Multi-lag DAGMA:** We construct explicit temporal input blockings $Z = [x(t-L), \ldots, x(t)]$ and apply DAGMA to learn a weight matrix whose blocks correspond to lag-specific temporal functional dependencies. This provides direct, interpretable evidence for which temporal lags carry predictive information, and places the single-graph GSL of our earlier study [7] as the contemporaneous special case ($L = 0$) of the formulation: that construction learned simultaneous per-sensor structure, not explicitly lagged structure (Section 5.3).
2. **T-GCN-MultiGSL-Mix:** We propose a T-GCN variant that processes each lag-specific graph separately and uses a per-node, per-timestep learned gate to combine the resulting representations through learned mixing.

Experiments on the Los-loop highway dataset demonstrate that T-GCN-MultiGSL-Mix achieves a mean RMSE improvement of 14.9% over a no-graph baseline

at 5-minute sampling, validated across five random seeds and confirmed against a parameter-matched control; at 15-minute sampling on the same network the improvement grows to 27.4%. A lag ablation study shows that all three temporal lags contribute complementary information. On the SZ-Taxi urban dataset, improvements are marginal, and we explicitly report this dataset dependence rather than claiming universal benefit.

The remainder of this paper is organized as follows. Section 2 reviews background on GCNs and T-GCNs. Section 3 presents the multi-lag DAGMA formulation and T-GCN-MultiGSL-Mix architecture. Section 4 describes the experimental setup. Section 5 presents results. Section 6 discusses findings. Section 7 states limitations. Section 8 concludes.

2 Background

2.1 Problem Formulation

We model the road network as a graph $G = (V, E)$, where $V = \{v_1, \ldots, v_N\}$ represents N road segments and $\mathbf{A} \in \{0, 1\}^{N \times N}$ is the binary adjacency matrix encoding road connectivity. Traffic data is represented by a feature matrix $X \in \mathbb{R}^{T \times N}$, where $X_t \in \mathbb{R}^N$ represents traffic speeds across all roads at time t. The forecasting task learns a mapping from historical observations to future traffic conditions:

$$[X_{t+1}, \ldots, X_{t+T}] = f(G; [X_{t-n}, \ldots, X_t]) \tag{2.1}$$

where n is the historical window size and T is the prediction horizon.

2.2 Graph Convolutional Networks

GCNs extend convolution to graph-structured data through a layer-wise propagation rule, due to Kipf and Welling [2]:

$$\mathbf{H}^{(l+1)} = \sigma\left(\tilde{\mathbf{D}}^{-\frac{1}{2}}\tilde{\mathbf{A}}\tilde{\mathbf{D}}^{-\frac{1}{2}}\mathbf{H}^{(l)}\mathbf{W}^{(l)}\right) \tag{2.2}$$

where $\tilde{\mathbf{A}} = \mathbf{A} + \mathbf{I}_N$ is the adjacency matrix with self-connections, $\tilde{\mathbf{D}}$ is the degree matrix, $\mathbf{W}^{(l)}$ is a trainable weight matrix, and $\sigma(\cdot)$ is a nonlinear activation. The normalized adjacency $\hat{A} = \tilde{\mathbf{D}}^{-\frac{1}{2}}\tilde{\mathbf{A}}\tilde{\mathbf{D}}^{-\frac{1}{2}}$ aggregates information from neighboring nodes while preserving feature scale. A two-layer GCN is:

$$f(\mathbf{A}, \mathbf{X}) = \sigma\left(\hat{A}\sigma\left(\hat{A}\mathbf{X}\mathbf{W}^{(1)}\right)\mathbf{W}^{(2)}\right) \tag{2.3}$$

2.3 Temporal Graph Convolutional Network

The T-GCN of Zhao et al. [3] combines GCN for spatial feature extraction with a Gated Recurrent Unit (GRU) for temporal modeling. At each timestep, the GCN extracts spatial features $\mathbf{Z}_t = f(\mathbf{A}, \mathbf{X}_t)$, which are then processed by the GRU to capture temporal dynamics. The GRU uses update gate $\mathbf{u}_t$, reset gate $\mathbf{r}_t$, and candidate

hidden state $\mathbf{c}_t$ with learnable weight matrices $\mathbf{W}_u$, $\mathbf{W}_r$, $\mathbf{W}_c$. The model is trained by minimizing:

$$\mathcal{L} = |\, Y_t - \widehat{Y}_t \,|_2 + \lambda \,|\, \Theta \,|_1 \tag{2.4}$$

where $\widehat{Y}_t$ is the prediction, Y_t is the ground truth, and λ controls L1 regularization.

3 Proposed Method

3.1 Motivation for Data-Driven Graph Structure Learning

In both GCN [2] and T-GCN [3], the adjacency matrix $\mathbf{A}$ is predefined based on physical road proximity. This has two fundamental limitations. First, physical connectivity is an imperfect proxy for functional traffic dependency: roads that are physically distant may exhibit strong traffic correlations due to commuting patterns, while adjacent roads may have minimal mutual influence. Second, dense physical graphs (e.g., 2833 edges for 207 nodes in the Los-loop dataset) can induce oversmoothing through excessive spatial aggregation in graph convolution layers.

Graph Structure Learning addresses these limitations by learning $\mathbf{A}$ directly from traffic data. We employ DAGMA [6], which extends the NOTEARS framework of Zheng et al. [5] for continuous DAG optimization.

3.2 DAGMA-Based Graph Structure Learning

DAGMA models the data through a structural equation model defined by a weighted adjacency matrix $W \in \mathbb{R}^{d\times d}$. Instead of operating on the discrete space of DAGs, it optimizes over continuous real matrices with a smooth acyclicity constraint $h(W) = 0$, where:

$$h(W) = \operatorname{tr}\left(e^{W\circ W}\right) - d = 0 \tag{3.1}$$

and $\circ$ denotes the Hadamard product. The optimization uses an augmented Lagrangian:

$$L^{\rho}(W,\alpha) = \text{score}(W) + \frac{\rho}{2}|h(W)|^2 + \alpha h(W) \tag{3.2}$$

where score(W) measures data fit, α is the Lagrange multiplier, and $\rho > 0$ is a penalty parameter. The algorithm iteratively updates W until $h(W) < \epsilon$, then applies thresholding: $\hat{W} = W \circ \mathbf{1}(|W| > \omega)$.

Convention: Throughout this paper, we follow the DAGMA convention where $W[i, j]$ represents a directed dependency from variable i to variable j.

Threshold semantics. The edge-support rule used throughout is the absolute-magnitude rule $A_{ij} = \mathbf{1}(|W_{ij}| > \tau)$: an edge survives if its estimated DAGMA coefficient survives the library's sparsity threshold, regardless of sign. This matches the threshold semantics of the DAGMA library itself and treats excitatory and inhibitory dependencies symmetrically; it is a methodological clarification rather than an empirical factor, since no negative coefficient survived the threshold on the datasets studied (Section 4.5).

3.3 Multi-Lag DAGMA Formulation

A single learned graph captures contemporaneous or aggregated dependencies but cannot distinguish between different temporal scales of traffic propagation. To address this, we construct an explicit multi-lag input representation.

Given traffic observations $x(t) \in \mathbb{R}^N$ and a lag window L, we define the multi-lag input:

$$Z_t = \big[x(t{-}L),\ x(t{-}L{+}1),\ \ldots,\ x(t{-}1),\ x(t)\big] \in \mathbb{R}^{(L+1)\times N} \tag{3.3}$$

which is flattened to a vector of dimension $(L+1)\cdot N$. The matrix Z is formed by stacking Z_t over all training time steps.

Applying DAGMA to Z produces a weight matrix $W \in \mathbb{R}^{(L+1)N\times(L+1)N}$. The block structure of W is:

- $W[l\cdot N : (l{+}1)\cdot N,\ L\cdot N : (L{+}1)\cdot N]$ represents dependencies from lag l to the current time step
- $W[L\cdot N : (L{+}1)\cdot N,\ L\cdot N : (L{+}1)\cdot N]$ represents contemporaneous dependencies

Since $W[i,j]$ means variable $i \to$ variable j, the block $W[l\cdot N+i,\ L\cdot N+j]$ captures the functional dependency from sensor i at time $t{-}(L{-}l)$ to sensor j at time t. This provides explicit, interpretable lag-specific temporal functional dependencies.

For each lag l, we extract the corresponding block and apply thresholding and self-loop removal to obtain a binary adjacency matrix:

$$A_l[i,j] = \mathbf{1}\big(|W[l\cdot N+i,\ L\cdot N+j]| > \tau\big)\cdot(1-\delta_{ij}) \tag{3.4}$$

where τ is a threshold and δ_{ij} is the Kronecker delta.

For $L=3$, this yields four blocks: lag_3 ($x(t{-}3)\to x(t)$), lag_2 ($x(t{-}2)\to x(t)$), lag_1 ($x(t{-}1)\to x(t)$), and current ($x(t)\to x(t)$). Empirically, these blocks exhibit substantially different edge structures with low cross-lag overlap, confirming that traffic dependencies are temporally heterogeneous.

We emphasize the distinction between the two DAGMA constructions used in this paper. The multi-lag fit above is the *explicitly lagged* construction: variables are the lag-stacked blocks of $Z_t = [x(t{-}L),\ldots,x(t)]$, so a block edge $i\to j$ directly encodes a lag-specific temporal dependency. The single-graph GSL baseline (Section 5.3, Appendix A) is instead a *contemporaneous* construction: DAGMA is applied to simultaneous per-sensor observations in a single N-variable fit, and the resulting DAG encodes present-observation functional dependencies without explicit temporal lag blocks. The temporal interpretation of the learned structure in this paper is carried entirely by the explicit multi-lag construction, not by the contemporaneous baseline.

3.4 T-GCN-MultiGSL-Mix

Standard T-GCN uses a single adjacency matrix for all timesteps. We propose T-GCN-MultiGSL-Mix, which processes each lag-specific graph separately and learned mixing of the results.

3.4.1 Architecture

Given K lag-specific graphs $\{A_1, \ldots, A_K\}$, we precompute their graph Laplacians $\{L_1, \ldots, L_K\}$. At each input timestep t:

1. The gate network computes per-node, per-graph weights:

$$\alpha_t = \text{softmax}\big(\text{MLP}([x_t; h_{t-1}])\big) \in \mathbb{R}^{N \times K} \tag{3.5}$$

   where $[x_t; h_{t-1}]$ concatenates the current input and previous hidden state.
2. A weighted adjacency is formed:

$$\tilde{L}_t = \sum_{k=1}^{K} \alpha_t^{(k)} \cdot L_k \in \mathbb{R}^{N \times N} \tag{3.6}$$

3. Graph convolution uses this mixed adjacency:

$$g_t = \tilde{L}_t \cdot [x_t; h_{t-1}] \tag{3.7}$$

4. The GRU update proceeds with g_t:

$$z_t = \sigma(W_z \cdot g_t) \tag{3.8}$$
$$r_t, u_t = \text{chunk}(z_t) \tag{3.9}$$
$$c_t = \tanh(W_n \cdot [x_t; r_t \odot h_{t-1}]) \tag{3.10}$$
$$h_t = u_t \odot h_{t-1} + (1 - u_t) \odot c_t \tag{3.11}$$

3.4.2 Key Properties

The gate operates at three levels simultaneously:

- **Per-node:** each of the N nodes has independent gate weights, allowing different sensors to use different lag graphs
- **Per-timestep:** the gate recomputes at each of the 12 input steps, adapting the graph weighting to the temporal context
- **Differentiable:** the gate is learned jointly with the forecasting objective via backpropagation

The learned graphs themselves are static: they are estimated once from training data and remain fixed during inference. What adapts over time is the model's *use* of them—the GRU models the temporal dynamics of traffic, and the gate selects, per node and per timestep, how the fixed lag-specific dependency structures are combined. In contrast to a fixed physical-proximity graph, the learned structure encodes functional traffic dependencies rather than road adjacency, and the temporal models then propagate these dependencies dynamically.

This contrasts with simpler multi-graph approaches evaluated in the same experiment artifacts (Section 5.1 reports the fixed-assignment variant; the learned-weight variant, T-GCN-MultiGSL-Weighted, achieves 4.710 on the same benchmark):

- **Union of lag graphs:** merges all lag graphs into one static adjacency, losing lag-specific information
- **T-GCN-MultiGSL-Weighted:** learns a fixed global weight per lag graph, without per-node or per-timestep adaptation
- **T-GCN-MultiGSL:** assigns graphs to timesteps in a fixed cyclic pattern

3.4.3 Parameter Count

For hidden dimension H and K lag graphs, T-GCN-MultiGSL-Mix has approximately $3H^2+6H+(H+1)H+H+(H+1)K+K$ parameters. With $H=64$ and $K=3$, this yields 17,091 parameters, compared to 12,672 for standard T-GCN.[1] A parameter-matched control (Section 5.4) confirms that this capacity difference does not explain the performance improvement.

4 Experimental Setup

4.1 Datasets

We evaluate on two widely-used traffic datasets:

- **SZ-Taxi:** Taxi trajectory data from Shenzhen, China (January 2015), with speed measurements from 156 sensors aggregated at 15-minute intervals. The physical graph has 532 edges.
- **Los-loop:** Real-time traffic speed data from 207 loop detectors on Los Angeles County highways (March 1–7, 2012), aggregated every 5 minutes. The physical graph has 2833 edges.

Both datasets are split into training (80%) and testing (20%) sets, preserving temporal order. Performance is evaluated across prediction horizons PH $\in \{1, 2, 3, 4\}$.

4.2 Multi-Lag DAGMA Configuration

For the multi-lag formulation, we use lag window $L = 3$, yielding input dimension $(L+1) \cdot N$:

- SZ-Taxi: $4 \times 156 = 624$ variables
- Los-loop: $4 \times 207 = 828$ variables

[1] Checkpoint-level parameter counts, which additionally include the output projection, are 17,156 and 12,737 respectively; the capacity difference between the two models is identical under either accounting.

DAGMA hyperparameters: $\lambda_1 = 0.01$, loss type = L2, warm-up iterations = 30,000, max iterations = 60,000. The DAGMA output is thresholded at $\tau = 0.1$ and self-loops are removed. DAGMA is applied only to training data; test data never enters the graph construction.

4.3 Temporal-Resolution Protocol (Los-loop at 15 Minutes)

To test whether the reported effects depend on the sampling interval, we additionally resample Los-loop to 15-minute resolution by averaging each consecutive block of three 5-minute steps, yielding $T = 672$ observations (537 train / 135 test). Normalization uses the training-data maximum only (feat_max = 70.0), as in all other experiments. DAGMA is computed once on the 15-minute training split (828 variables, identical λ_1) and the resulting lag graphs are reused across all horizons and seeds. In this variant, PH=k denotes $k \times 15$ minutes ahead; RMSE magnitudes are therefore not directly comparable to the 5-minute tables, and we compare relative improvements.

4.4 Forecasting Models

We compare the following T-GCN variants:

- **T-GCN-NoSpatial:** T-GCN with identity adjacency (no spatial information). This is the critical baseline—it tests whether any graph helps.
- **Physical:** T-GCN with the predefined road network adjacency.
- **T-GCN-MultiGSL:** T-GCN that assigns lag-specific graphs to input timesteps in a fixed pattern (corrected alignment).
- **T-GCN-MultiGSL-Mix:** Our proposed method with per-node, per-timestep learned mixing over lag-specific graphs.

All models use hidden dimension $H = 64$, Adam optimizer (learning rate 0.001, weight decay 0.0001), batch size 128, historical window $n = 12$, and 50 training epochs. The loss function is MSE with L1 regularization. Results are reported as mean $\pm$ standard deviation over 5 random seeds (seeds 42–46) for the main comparison.

4.5 Reproducibility

Graph construction and forecasting are separately reproducible. DAGMA-linear is deterministic (zero initialization, no internal randomness): a dedicated determinism audit verified bit-identical outputs across repeated runs and processes, with differences at approximately one ULP across thread counts and identical thresholded support. Training-time randomness is isolated by the seed policy: every reported table states its seed count, and the main comparisons use seeds 42–46 with population-standard-deviation reporting (mean$\pm$std). The learned graphs are thresholded by absolute magnitude inside the DAGMA fit (library semantics; Section 3); no negative coefficient survived the threshold on either dataset, so the support rule is empirically equivalent to a positive-only rule here. The original submission's graph artifacts were archived and re-derived from the original data-loading code; the revised protocol's slice-0 fits reproduce the historical slice-0 graph supports exactly at PH=1, 2, and 4

Table 1: Oversmoothing evidence: Los-loop (PH=1, seed=42). Dense graphs—whether physical or learned from correlations—produce substantially worse RMSE than no graph at all, isolating density (not the graph's origin) as the harmful factor. Sparse learned graphs and multi-lag graphs improve over T-GCN-NoSpatial. Sparse controls (bottom block) are five-seed mean±std at a matched 30-edge budget; other rows are seed-42 values.

Graph	Edges	RMSE	MAE
Physical (207 nodes)	2833	7.658	5.329
Corr-K=8 (learned correlations)	1656	6.915	4.540
T-GCN-NoSpatial (identity)	207	5.143	3.028
Single DAGMA (τ=0.3)	6	5.213	3.192
Single DAGMA (τ=0.1)	60	6.057	3.693
T-GCN-MultiGSL	30	4.715*	2.755
T-GCN-MultiGSL-Mix	30	4.458	2.696
Matched-budget sparse controls (five seeds, mean±std, same 30-edge budget):			
Random sparse (RandTop30)	30	6.096±0.107	3.756±0.150
Top-correlation sparse (CorrTop30)	30	5.389±0.088	3.247±0.056

* Stage-26 evaluation run; the validation rerun of the identical configuration gives 4.717 (Table 4). Both are reported for transparency.

(26/28 at PH=3), with shared-edge weights agreeing to within approximately 0.016 (Section 5.3). Detailed provenance of every graph artifact—input construction, λ_1, threshold, sign counts, and software versions—is documented in the project repository.

4.6 Evaluation Metrics

We report Root Mean Squared Error (RMSE) and Mean Absolute Error (MAE):

$$\text{RMSE} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(Y_i - \hat{Y}_i)^2}, \quad \text{MAE} = \frac{1}{n}\sum_{i=1}^{n}|Y_i - \hat{Y}_i| \tag{4.1}$$

where Y_i and $\hat{Y}_i$ are actual and predicted traffic speeds. RMSE is the primary metric.

5 Results

5.1 Dense Physical Graphs and Oversmoothing

A fundamental finding across our experiments is that dense physical graphs degrade T-GCN performance. Table 1 compares the physical graph against a no-graph baseline and sparse learned graphs on Los-loop (PH=1; seed-42 values except the matched-budget controls, which are five-seed means).

The physical graph (2833 edges) produces RMSE of 7.658, while T-GCN-NoSpatial (identity adjacency) achieves 5.143—a 32.8% improvement from removing the graph

entirely (single seed). This confirms that dense spatial aggregation hurts forecasting. The effect is driven by density rather than by the physical origin of the graph: a dense *learned* correlation graph with 1656 edges (Corr-K=8, each node connected to its eight most correlated sensors) also degrades to 6.915, close to the physical graph's 7.658. Only when the graph is sparse do we see gains: the sparse T-GCN-MultiGSL and T-GCN-MultiGSL-Mix approaches further improve over T-GCN-NoSpatial, demonstrating that carefully structured sparse graphs provide genuine benefit.

Is sparsity alone responsible? Matched-budget controls.

A natural concern is whether the advantage of the learned graphs merely reflects their sparsity—fewer edges means less oversmoothing—rather than the specific learned structure. We therefore trained T-GCN on two sparse control graphs at exactly the same 30-edge budget as T-GCN-MultiGSL, over five seeds (Table 1, bottom block): a random sparse graph (RandTop30; 30 edges placed uniformly at random) and a top-correlation graph (CorrTop30; the 30 largest absolute cross-sensor Pearson correlations computed on training data only). Neither control reproduces the learned-graph gain. The random sparse graph is markedly *worse* than no graph at all (6.096±0.107 vs. 5.234±0.090), and the correlation-based placement (5.389±0.088) is also no better than T-GCN-NoSpatial. Two further controls sharpen the conclusion. First, assembling the same DAGMA-derived lag edges into a single union adjacency (28 edges) used by a standard T-GCN cell yields 5.928 (PH=1, seed 42)—worse than no graph—so the edges alone, without per-lag use, do not produce the gain. Second, a thresholded single-DAG graph (60 edges, Table 1) also underperforms. Sparsity is therefore necessary but not sufficient: at a matched 30-edge budget, neither a random sparse graph nor a top-correlation graph matches T-GCN-NoSpatial, whereas the same number of DAGMA lag-block edges yields 4.794 (T-GCN-MultiGSL) and 4.452 (T-GCN-MultiGSL-Mix) as five-seed means. The benefit arises from the *combination* of DAGMA-learned lag-specific edges and their per-lag use in the multi-graph architecture, not from sparsity alone.

5.2 From Single-Graph GSL to Multi-Lag Structure

The original submission's GSL/cGSL experiments (Appendix A) established that a sparse DAGMA-learned graph substantially outperforms the dense physical adjacency for both GCN and T-GCN, with a striking asymmetry: the symmetrized cyclic variant (cGSL) was optimal for the purely spatial GCN, while the acyclic DAG was optimal for the temporally equipped T-GCN. On Los-loop, TGCN-GSL reduced RMSE by a mean 21.8% across horizons PH=1–4 relative to the physical graph (e.g., PH=1: 6.588 → 4.818), a single-seed result whose full tables and protocol are retained in Appendix A. Under the revised training pipeline this baseline is independently reproduced at five seeds (21.7% mean improvement, Section 5.3); absolute RMSE values are not directly comparable between the two pipelines. That asymmetry raised the question this revision answers: *what does a single learned DAG actually encode?* Two controls were missing in the original study. First, a no-graph baseline, which reveals that the dense

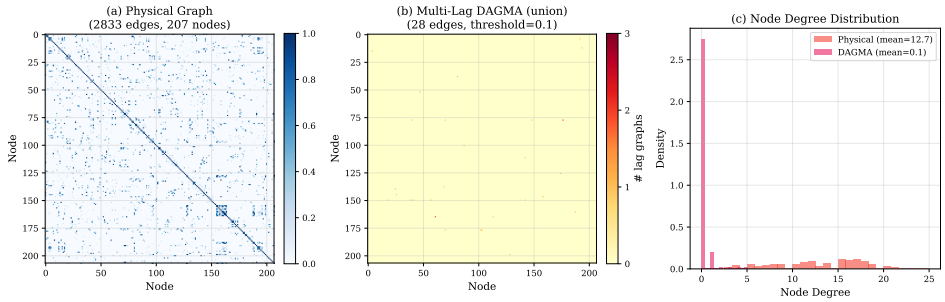


Fig. 1: Physical versus learned graph structure (Los-loop). (a) Physical adjacency matrix with 2833 edges (including 207 diagonal self-connections). (b) Multi-lag DAGMA union graph ($\tau = 0.1$): 30 lag-specific edges across the three lag graphs, 28 distinct off-diagonal edges. (c) Node degree distribution (excluding self-connections): physical graph has mean degree 12.7, while the DAGMA union graph has mean degree 0.14.

Table 2: Multi-lag DAGMA block statistics (Los-loop, $L = 3$, $\tau = 0.1$). Different lag blocks capture different dependency structures. Edge counts include self-loops; the corresponding cross-sensor training graphs (self-loops removed) retain 12, 3, and 15 edges for lag_1, lag_2, and lag_3 respectively (30 in total).

Block	**Interpretation**	**Edges**	**Max $\|w\|$**	**Density**
current	$x(t) \to x(t)$	70	0.653	0.001634
lag_1	$x(t-1) \to x(t)$	90	0.804	0.002100
lag_2	$x(t-2) \to x(t)$	5	0.673	0.000117
lag_3	$x(t-3) \to x(t)$	16	0.296	0.000373

physical graph itself is harmful (Section 5.1). Second, an explicit temporal decomposition of the learned structure, which the multi-lag formulation provides—converting a post-hoc interpretation of one ambiguous matrix into directly measurable, structurally distinct lag-specific graphs.

The multi-lag DAGMA formulation ($L = 3$) produces lag-specific graphs with substantially different structures. Table 2 reports the statistics for Los-loop at threshold $\tau = 0.1$.

The current block has 70 cross-sensor edges, lag_1 has 90 edges (dominated by self-loops), lag_2 has only 5 edges, and lag_3 has 16 edges. These blocks are structurally distinct, confirming that traffic dependencies are temporally heterogeneous.

5.3 The Single-Graph GSL Baseline Revisited

Before evaluating the multi-lag formulation, we re-establish the single-graph GSL baseline of the original submission under the revised training pipeline, with full provenance.

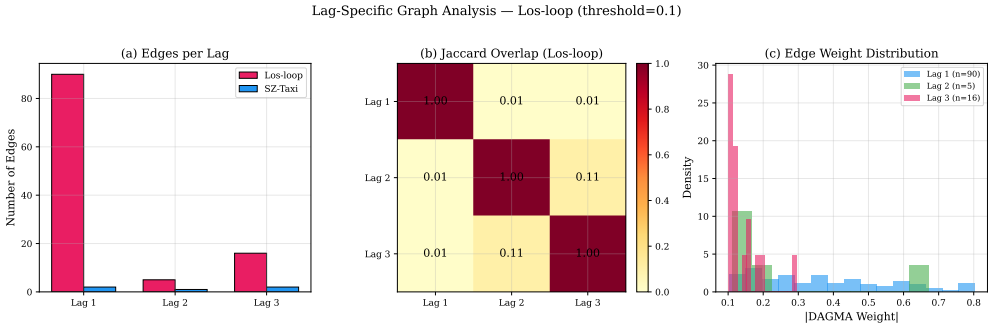


Fig. 2: Lag-specific graph analysis. (a) Edge count per lag for Los-loop and SZ-Taxi ($\tau = 0.1$). (b) Jaccard overlap between lag-specific edge sets (Los-loop): low overlap confirms structural distinctness. (c) DAGMA weight distributions per lag.

Table 3: Revised-protocol single-graph GSL baseline on Los-loop (five seeds 42–46, mean±std). The T-GCN-GSL row uses a 28-edge contemporaneous DAGMA DAG per horizon; Physical is the dense road-network graph (2833 edges). Absolute RMSE values are not directly comparable to the original submission's tables (Appendix A) because the training pipeline was revised.

Method	**PH=1**	**PH=2**	**PH=3**	**PH=4**
Physical (2833 edges)	7.772±0.140	8.118±0.189	8.456±0.047	8.554±0.169
T-GCN-GSL (28 edges)	5.792±0.196	6.328±0.166	6.669±0.080	6.999±0.084
Improvement	**25.5%**	22.0%	21.1%	18.2%

The protocol is the contemporaneous construction of Section 3.3: for each horizon PH, DAGMA is fitted once to the offset-0 subsampled training windows (a genuine single DAG per horizon; $\lambda_1 = 0.02$, L2 loss, sparsity threshold $\omega = 0.3$ applied inside the fit), and the resulting 28-edge DAG is used with standard T-GCN. Both baselines are evaluated over five seeds (42–46).

As Table 3 shows, the sparse learned DAG outperforms the dense physical graph at every horizon, with a mean improvement of 21.7% across PH=1–4 (25.5% at PH=1). The original submission reported this comparison as a single-seed 26.9% PH=1 improvement and a 21.8% multi-horizon mean under its original pipeline (Appendix A); the revised protocol reproduces both findings in relative terms at five seeds (25.5% and 21.7%), with non-overlapping $\pm 1\sigma$ bands at every horizon. Absolute RMSE values differ between the two protocols because the training pipeline itself was revised, so the two sets of numbers must not be compared directly.

The reproduction extends to the learned graphs themselves. The original submission's DAGMA fits were recovered from the archived artifacts and re-derived from the original data-loading code, which fitted one DAGMA model per offset $i \in \{0, \ldots, \text{PH}-1\}$ on stride-PH subsampled windows and unioned the K resulting supports. The revised protocol's first-offset (slice-0) fits reproduce the original slice-0

Table 4: Multi-seed validation on Los-loop (PH=1). T-GCN-MultiGSL-Mix outperforms T-GCN-NoSpatial in all five seeds. Mean improvement: 14.9%.

Method	S42	S43	S44	S45	S46	Mean±Std
T-GCN-NoSpatial	5.143	5.281	5.386	5.205	5.154	5.234±0.090
T-GCN-MultiGSL	4.717	4.737	4.752	4.994	4.771	4.794±0.102
T-GCN-MultiGSL-Mix	**4.458**	**4.660**	**4.335**	**4.261**	**4.547**	**4.452±0.143**

Table 5: Parameter-matched control (Los-loop, PH=1, seed=42). Adding 35% more parameters to T-GCN-NoSpatial improves RMSE by only 0.1%. T-GCN-MultiGSL-Mix's improvement is from the gating architecture.

Model	hidden_dim	Parameters	RMSE
T-GCN-NoSpatial	64	12,672	5.143
T-GCN-NoSpatial (larger)	74	16,872	5.137
T-GCN-MultiGSL-Mix	64	**17,091**	**4.458**

graphs at the support level: exact 28/28 edge-set agreement at PH=1, 2, and 4, 26/28 at PH=3, with shared-edge weights agreeing to within approximately 0.016 (not bit-identical, owing to the float64→float32 data path and two years of library/environment drift). The historical as-used graphs additionally unioned the K offset fits, yielding 28/32/33/39 edges for PH=1–4; the revised single-DAG-per-horizon graphs are strictly contained in those unions and match the manuscript's own description of the method ("a single $N \times N$ weight matrix W"). Appendix A documents both protocols.

5.4 T-GCN-MultiGSL-Mix: Multi-Seed Validation and Parameter Control

Table 4 presents the primary result: Los-loop, PH=1, five random seeds.

T-GCN-MultiGSL-Mix beats T-GCN-NoSpatial in all 5 seeds with a mean RMSE of 4.452 versus 5.234, corresponding to a 14.9% improvement (five-seed mean; the seed-42-only comparison is 13.3%). The fixed multi-graph approach also consistently outperforms T-GCN-NoSpatial (8.4% improvement), but T-GCN-MultiGSL-Mix provides an additional 7.1% improvement through learned graph mixing.

To rule out the explanation that T-GCN-MultiGSL-Mix's improvement comes from having more parameters, we compare against a T-GCN-NoSpatial with matched capacity (Table 5).

The parameter-matched T-GCN-NoSpatial (16,872 parameters) achieves RMSE 5.137, virtually identical to the standard T-GCN-NoSpatial (5.143). T-GCN-MultiGSL-Mix (17,091 parameters) achieves 4.458. Thus 99% of the improvement comes from the gating architecture, not from additional capacity.

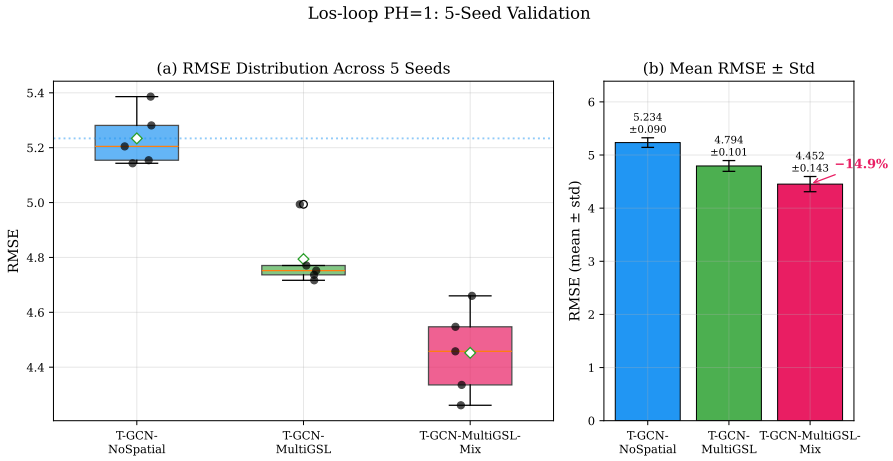


Fig. 3: Multi-seed validation on Los-loop (PH=1, seeds 42–46). (a) Box plot: T-GCN-MultiGSL-Mix consistently outperforms T-GCN-NoSpatial across all seeds. (b) Mean RMSE ± std: T-GCN-MultiGSL-Mix achieves 4.452±0.143 versus T-GCN-NoSpatial 5.234±0.090 (14.9% improvement).

Predicted vs. actual behaviour.

Figure 4 shows one-step-ahead predictions for the three most variable Los-loop nodes over a 100-step test window (PH=1, seed=42). Both models follow the ground truth closely—the overall Pearson correlation between predictions and observations across the full test set is $r = 0.95$ for T-GCN-MultiGSL-Mix and $r = 0.93$ for T-GCN-NoSpatial, with per-node values reaching $r \approx 0.99$—and both exhibit the smooth, slightly lagged character typical of MSE-trained predictors that approximate conditional means. On two of the three displayed nodes, T-GCN-MultiGSL-Mix tracks rapid changes more faithfully than T-GCN-NoSpatial (windowed r of 0.95 vs. 0.92 and 0.96 vs. 0.94), consistent with its lower RMSE; the remaining node is a low-signal case on which both models correlate weakly with the truth.

5.5 Ablation: Lags and Prediction Horizons

Table 6 shows the contribution of each temporal lag.

Each individual lag provides approximately 10% improvement over T-GCN-NoSpatial. The best two-lag combination is lag_1+lag_2 (11.4%), and the full three-lag configuration achieves 13.3%. This confirms that all three temporal scales carry complementary predictive information.

Table 7 reports results across all four prediction horizons on Los-loop (seed 42; the five-seed mean±std for Physical and the single-graph GSL baseline is given in Table 3).

On Los-loop, T-GCN-MultiGSL-Mix consistently outperforms T-GCN-NoSpatial across all horizons, with the largest relative improvement at PH=1 (13.3%). The advantage persists but narrows at longer horizons. The dependence of performance on the DAGMA threshold τ—and hence on the learned sparsity level—is reported in

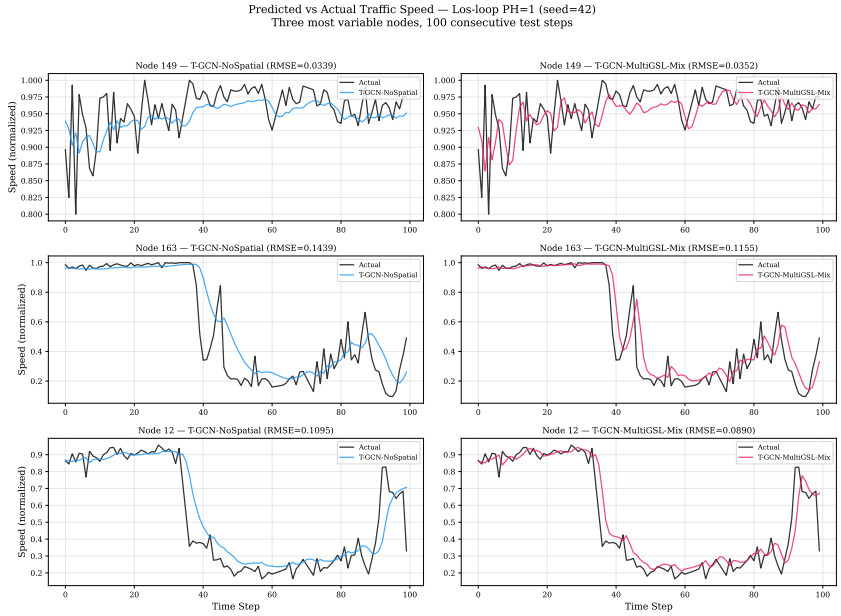


Fig. 4: Predicted vs. actual traffic speed for three high-variance nodes on Los-loop (PH=1, seed=42), 100 consecutive test steps. Both T-GCN-NoSpatial and T-GCN-MultiGSL-Mix predictions closely track the ground truth, with the smooth, slightly lagged character typical of MSE-trained models. Per-node windowed correlations are given in the text.

Table 6: Lag ablation (Los-loop, PH=1, seed=42). All three lags contribute; the full combination is best.

Configuration	Edges	RMSE	Improvement
T-GCN-NoSpatial	207	5.143	—
lag_2 only	3	4.619	10.2%
lag_3 only	15	4.605	10.5%
lag_1 only	12	4.605	10.5%
lag_1+lag_3	27	4.646	9.7%
lag_2+lag_3	18	4.638	9.8%
lag_1+lag_2	15	4.559	11.4%
All 3 lags	**30**	**4.458**	**13.3%**

Appendix C: T-GCN-MultiGSL-Mix achieves its best RMSE near the operating point of 30 edges.

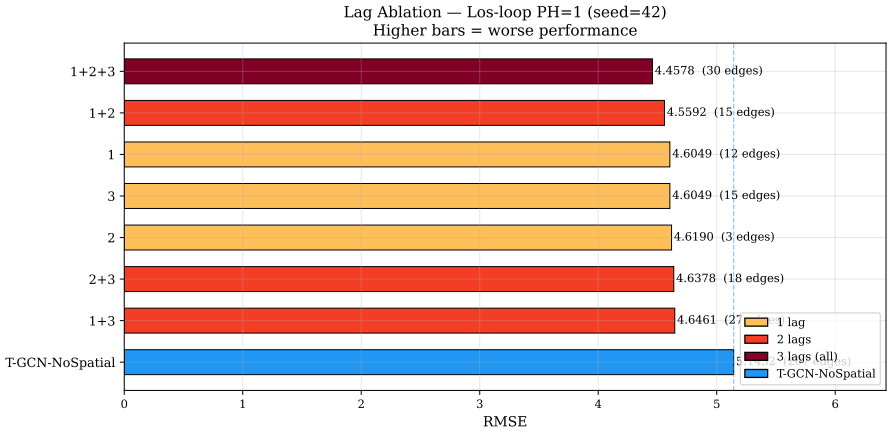


Fig. 5: Lag ablation study (Los-loop, PH=1, seed=42). Each lag provides ∼10% improvement individually; all three lags together achieve 13.3%. Color intensity reflects number of lags used.

Table 7: Multi-horizon results on Los-loop (seed=42). T-GCN-MultiGSL-Mix consistently outperforms T-GCN-NoSpatial and Physical across all horizons.

Method	PH=1	PH=2	PH=3	PH=4
Physical (2833 edges)	7.658	8.002	8.512	8.540
T-GCN-GSL (28 edges)	5.548	6.156	6.645	6.932
T-GCN-NoSpatial	5.143	5.642	6.164	6.502
T-GCN-MultiGSL	4.715	5.549	5.934	6.336
T-GCN-MultiGSL-Mix	**4.458**	**5.308**	**5.687**	**6.004**

5.6 Robustness Across Temporal Resolution

Does the effect survive a change of sampling interval on the same sensor network? We resample Los-loop to 15-minute intervals (three-step averaging, $T = 672$) and repeat the full pipeline—DAGMA learned once on the 15-minute training split, then reused across horizons. Table 8 reports five-seed results.

At 15-minute sampling the relative improvement over T-GCN-NoSpatial is *larger* than at 5-minute sampling: +27.4% at PH=1 versus +14.9% (five-seed mean), with T-GCN-MultiGSL-Mix winning in all five seeds (per-seed improvements of +23.2% to +31.3%) and the advantage persisting across all four horizons. This shows that the multi-lag benefit is not an artifact of one sampling interval; the implications for the dataset-dependence analysis below are discussed in Section 6.

Table 8: Los-loop at 15-minute sampling (seeds 42–46, mean±std). PH=k denotes $k \times 15$ minutes ahead: 15, 30, 45, and 60 minutes for $k = 1$–4. RMSE magnitudes are not directly comparable with the 5-minute tables above, so we compare relative improvements. The lag graphs (32 cross-sensor edges over three lags) come from a single PH-independent DAGMA run on the 15-minute training split.

Method	PH=1	PH=2	PH=3	PH=4
T-GCN-NoSpatial	8.600±0.249	9.311±0.197	10.048±0.074	10.460±0.196
T-GCN-MultiGSL	7.246±0.298	8.571±0.327	9.112±0.077	9.756±0.210
T-GCN-MultiGSL-Mix	**6.240±0.187**	**7.322±0.338**	**8.063±0.203**	**8.777±0.326**
Mix improvement	**+27.4%**	+21.4%	+19.8%	+16.1%

Table 9: SZ-Taxi multi-horizon results (five seeds 42–46, mean±std). Values are given to four decimal places because the between-method differences are small. The gated T-GCN-MultiGSL-Mix shows a marginal, directionally consistent advantage at PH=1–3 (winning in 5/5 paired seeds at each horizon) and no stable advantage at PH=4 (3/5 seeds); the ungated T-GCN-MultiGSL does not improve over T-GCN-NoSpatial. The oversmoothing pattern replicates on this dataset: the dense physical graph is again far worse than no graph (seed-42 RMSE 5.267 vs. 4.116).

Method	PH=1	PH=2	PH=3	PH=4
T-GCN-NoSpatial	4.1192±0.0069	4.1623±0.0055	4.1884±0.0011	4.2196±0.0013
T-GCN-MultiGSL	4.1302±0.0152	4.1600±0.0036	4.1988±0.0121	4.2273±0.0092
T-GCN-MultiGSL-Mix	**4.1091±0.0052**	**4.1515±0.0026**	**4.1804±0.0052**	4.2159±0.0073

5.7 Dataset Dependence: the SZ-Taxi Boundary

Table 9 reports the SZ-Taxi results over five seeds, promoted here from the appendix because the boundary of the method is as informative as its successes.

On SZ-Taxi the improvement is marginal and dataset-dependent. The gated T-GCN-MultiGSL-Mix improves over T-GCN-NoSpatial by only 0.19–0.26% at PH=1–3, although the direction is consistent, with Mix winning in 5/5 paired seeds at each of these horizons. At PH=4 the difference (+0.09%) is not stable: Mix wins in only 3/5 seeds, with per-seed deltas straddling zero, so we describe the PH=4 result as within noise rather than as an improvement. The ungated T-GCN-MultiGSL variant does not help on this dataset (its RMSE is equal to or worse than T-GCN-NoSpatial at three of the four horizons), indicating that the small gain comes from the gated mixing mechanism operating on the learned lag graphs, not from the graphs alone. No significance tests are claimed for these comparisons; we report means, standard deviations, and paired win counts only. This is not explained by temporal resolution: SZ-Taxi and the 15-minute Los-loop variant share the sampling interval, yet Los-loop shows strong improvements at both 5-minute and 15-minute sampling (Sections 5.4 and 5.6), so the

divergence tracks the dataset rather than the sampling interval. A candidate mechanism consistent with the evidence is graph learnability: at the operating threshold, DAGMA retains only 2 cross-sensor lag edges for SZ-Taxi versus 30–32 for Los-loop, suggesting that the urban network's dependency structure at 15-minute aggregation offers the learner far less exploitable signal. Other dataset differences (156 vs. 207 sensors, urban vs. highway dynamics) may also contribute. We therefore report SZ-Taxi as a dataset-dependent boundary of the method rather than claiming universal benefit.

6 Discussion

6.1 Why Dense Physical Graphs Fail

The most striking finding is that removing the physical graph entirely improves RMSE by 32.8% (Table 1). This oversmoothing effect arises because the physical graph's high density (2833 edges for 207 nodes) causes repeated graph convolution operations to aggregate information from increasingly distant and irrelevant nodes, washing out the distinctive features of individual road segments. This finding is consistent across both datasets and aligns with theoretical predictions about over-squashing in graph neural networks.

6.2 Why Multi-Lag Processing Helps

The multi-lag formulation provides two distinct advantages. First, it makes the temporal dependency structure explicit: instead of learning a single ambiguous graph, we obtain separate graphs for each temporal lag, each with a clear interpretation (Table 2). Second, the T-GCN-MultiGSL-Mix architecture can process these graphs through separate T-GCN cells and combine the results through learned mixing, preserving the lag-specific information that would be lost by merging into a single adjacency.

The fact that T-GCN-MultiGSL (same edges, fixed assignment) achieves 4.794 while T-GCN-MultiGSL-Mix achieves 4.452 demonstrates that *how* the lag-specific graphs are processed matters as much as *which* graphs are used. The 0.342 RMSE improvement from learned graph mixing represents a 7.1% further reduction beyond the fixed multi-graph approach. Training converges stably within 50 epochs for all methods, with T-GCN-MultiGSL-Mix reaching the lowest final training loss (Appendix C).

6.3 Why the Effect is Dataset-Dependent

The improvement on SZ-Taxi is marginal (0.19–0.26% at PH=1–3, five-seed mean, with the gated variant winning 5/5 paired seeds; no stable difference at PH=4) compared to the strong improvement on Los-loop (14.9% five-seed mean). Two hypotheses can be ruled out. First, temporal resolution is not the explanation: SZ-Taxi and the 15-minute Los-loop variant share the same sampling interval, yet Los-loop improves strongly at both 5-minute and 15-minute sampling (Sections 5.4 and 5.6), so the divergence tracks the dataset rather than the sampling interval. Second, capacity is not the explanation (Section 5.4). A candidate mechanism consistent with the evidence is graph learnability: at the operating threshold, DAGMA retains only 2 cross-sensor

lag edges for SZ-Taxi versus 30–32 for Los-loop, suggesting that the urban network's dependency structure offers the learner far less exploitable signal than highway propagation patterns, which have clear temporal structure (vehicles traveling at highway speeds create predictable lagged dependencies). Consistent with this, the ungated T-GCN-MultiGSL variant—which relies on the lag graphs alone—does not improve over T-GCN-NoSpatial on SZ-Taxi, whereas on Los-loop the same variant yields an 8.4% improvement: the gating mechanism partially compensates where the learned graphs carry little structure, but cannot manufacture signal that the graph learner did not find. Other dataset differences (156 vs. 207 sensors, urban vs. highway dynamics) may also contribute. This dataset dependence is an honest finding that the paper explicitly reports rather than hiding.

6.4 Sparsity Versus Learned Structure

The matched-budget controls of Section 5.1 allow a precise statement of what the learned graph contributes. Sparsity per se is not beneficial: a random 30-edge graph degrades RMSE by 16.5% relative to no graph, and a top-correlation placement of the same budget is no better than no graph. Nor is the gain explained by the DAGMA edges alone: the same edges assembled into a single union adjacency give 5.928, worse than no graph. The only 30-edge configuration that outperforms T-GCN-NoSpatial is the DAGMA lag-block edge set used per-lag inside the multi-graph architecture. We therefore attribute the improvement to the combination of DAGMA-learned lag-specific structure and the per-lag multi-graph processing, not to sparsity or to the presence of any graph.

6.5 Connection to Original GSL/cGSL Findings

The original GSL/cGSL experiments established that learned sparse graphs outperform dense physical graphs, and that different backbone architectures (GCN vs T-GCN) benefit from different graph structures. These findings remain valid and motivated the deeper investigation into temporal heterogeneity. The multi-lag formulation can be viewed as a principled extension: rather than debating whether the learned graph should be cyclic or acyclic, we construct lag-specific graphs that naturally capture different temporal aspects of the dependency structure. The reproduction analysis (Section 5.3) additionally established that the original single-graph construction was contemporaneous—DAGMA applied to simultaneous per-sensor observations—so the temporal reading of that original DAG is now replaced by the explicit lag-specific construction, and the original baseline result itself is independently reproduced at five seeds under the revised pipeline (21.7% mean improvement, Appendix A).

7 Limitations

This study has several limitations that delineate the boundaries of the current evidence:

1. **Dataset dependence:** The strong improvement (14.9% at 5-minute and 27.4% at 15-minute sampling) is observed on the Los-loop highway dataset, while the SZ-Taxi urban dataset shows only marginal benefit for the gated model (0.19–0.26% at PH=1–3, not stable at PH=4) and no clear benefit for the ungated multi-graph variant. Temporal resolution is ruled out as the explanation, since Los-loop improves strongly at both sampling intervals; the method's effectiveness appears to depend on the underlying traffic dynamics, plausibly on how much exploitable dependency structure the learner can discover.
2. **Two datasets only:** Evaluation is limited to two traffic datasets. Broader validation on diverse traffic networks (e.g., different cities, different sensor types, different time periods) would strengthen generalizability claims.
3. **DAGMA scalability:** Fitting time is substantial even at modest network sizes. The contemporaneous single-graph fits used for the GSL baseline take approximately 16 minutes per horizon for the 207-node Los-loop network on 4 CPU cores; archived 156-node SZ-Taxi audit fits took approximately 52–76 minutes each; and the multi-lag formulation increases the input dimension to $(L+1) \cdot N$, yielding an 828×828 fit that requires approximately 4 hours. Scaling to networks with thousands of sensors would require more efficient graph learning algorithms.
4. **Fixed lag window:** The current formulation uses a fixed lag window $L = 3$. The optimal lag structure may vary across datasets and traffic conditions. Learned lag selection remains future work.
5. **No λ sensitivity analysis:** The DAGMA sparsity hyperparameter was fixed at values used throughout ($\lambda_1 = 0.02$ for the single-graph protocol, 0.01 for the multi-lag fits) and no systematic λ sweep was performed. Sensitivity of the learned graphs and of forecasting performance to λ_1 was therefore not measured, and the reported graphs should be understood as one operating point rather than an optimized configuration.
6. **Static learned graphs:** The lag-specific graphs are learned once from training data and remain fixed during inference. While the GRU hidden state provides temporal dynamics and the gate provides per-timestep graph selection, the underlying dependency graphs themselves do not adapt to changing traffic conditions.
7. **Limited backbone architectures:** We evaluate only the studied T-GCN/GCN-family backbones. Whether the multi-lag approach benefits other graph-based forecasting architectures (e.g., ST-GCN, Graph WaveNet) remains to be tested; broader backbone validation is future work.
8. **No causal identification:** The DAGMA formulation discovers temporal functional dependencies, not causal relationships. The graph analyses in this paper are descriptive structural analyses of learned dependency structure, and we do not claim that the learned graphs represent causal traffic mechanisms or that they have been causally validated.
9. **Prediction horizons:** The main 5-minute experiments cover horizons PH=1–4; horizon counts beyond 4 (PH5–PH8) were not evaluated and remain future work. The 15-minute-sampling variant provides evidence up to 60 minutes ahead (PH=4 at 15-minute resolution), and its growing relative improvements with

horizon suggest that structure quality matters more at longer wall-clock horizons, but this does not substitute for directly evaluated longer horizons in the 5-minute setting.

8 Conclusion

This paper addresses two fundamental challenges in graph-based traffic forecasting: the oversmoothing caused by dense physical graphs, and the temporal heterogeneity of traffic dependencies that a single static graph cannot capture.

Our key findings are:

1. Dense physical road-network graphs can severely degrade GCN/T-GCN forecasting performance through excessive spatial aggregation. On the Los-loop dataset, removing the physical graph entirely improves RMSE by 32.8%.
2. Traffic dependencies are temporally heterogeneous: different temporal lags exhibit distinct functional dependency structures with low cross-lag overlap.
3. Multi-lag DAGMA explicitly discovers these lag-specific dependencies by constructing temporal input blockings $Z = [x(t-L), \ldots, x(t)]$ and extracting separate functional graphs for each lag.
4. T-GCN-MultiGSL-Mix exploits these lag-specific graphs through per-node, per-timestep learned mixing, achieving a mean RMSE improvement of 14.9% over a no-graph baseline on Los-loop across five random seeds at 5-minute sampling, and 27.4% at 15-minute sampling on the same network.
5. The improvement is robust: it holds in all five seeds at both sampling intervals, survives a parameter-matched control, and persists across all four prediction horizons.
6. The effect is dataset-dependent: on SZ-Taxi, the gated model's improvement is marginal (0.19–0.26% at PH=1–3, winning 5/5 paired seeds) and not stable at PH=4, while the ungated multi-graph variant does not help, indicating that the multi-lag approach is most effective when traffic networks exhibit strong lag-specific propagation patterns. Temporal resolution is ruled out as the explanation, as Los-loop improves strongly at both 5-minute and 15-minute sampling.
7. The gain is attributable to learned lag-specific structure rather than sparsity: at a matched 30-edge budget, random and correlation-based sparse graphs do not beat a no-graph baseline, and the same DAGMA edges used as a single union adjacency do not reproduce the multi-graph result.

The single-graph GSL baseline of the original submission was independently reproduced under the revised training pipeline (21.7% mean improvement over the physical graph across five seeds), with the historically submitted slice-0 graphs recovered at the support level; the original construction is now understood as contemporaneous, and the explicitly lagged multi-lag formulation supplies the temporal structure previously read into it.

Future work should explore learned lag window selection, sensitivity to the sparsity hyperparameter λ_1, sliding-window DAGMA for time-varying graph updates,

scalability to larger networks, longer prediction horizons, and integration with other graph-based forecasting architectures.

Acknowledgements. The author would like to thank Amin Amiri and Morteza Mostafazadeh for their valuable collaboration in converting the T-GCN codebase from PyTorch Lightning to pure PyTorch implementation.

Appendix A Original GSL/cGSL Results

This appendix presents the original Graph Structure Learning (GSL) and cyclic GSL (cGSL) results that motivated the multi-lag investigation. Method naming in this appendix follows the original submission; all tables in this appendix are **historical single-seed results** produced under the original study's protocol and pipeline. They are retained for transparency and because the GSL/cGSL backbone asymmetry they document remains scientifically informative; they are *not* current canonical results and are not directly comparable in RMSE magnitude to the main-text tables, which use the revised training pipeline. The main-text re-establishment of this baseline under the revised protocol is given in Section 5.3.

A.1 Method

The original GSL approach applies DAGMA to contemporaneous traffic observations $X \in \mathbb{R}^{T\times N}$, where each row is a single-timestep snapshot of all N sensor readings. This produces a single $N \times N$ weight matrix W. The GSL variant uses the binary thresholded DAG adjacency: $A_{GSL} = \mathbf{1}(|W| > \omega)$. The cGSL variant symmetrizes: $A_{cGSL} = A_{GSL} + A_{GSL}^T$.

Two protocol details, established during the revision by re-deriving the construction from the original data-loading code, complete the historical description. First, in the as-submitted pipeline DAGMA was not fitted once per horizon: for prediction horizon PH, the loader fitted $K = \text{PH}$ separate DAGMA models on stride-K offset-stratified subsamples of the training windows ($X = \text{data}[i :: K]$ for $i = 0, \ldots, K-1$) and used the *union* of the K resulting supports. The resulting as-used Los-loop graphs had 28/32/33/39 edges for PH=1–4 (the union of several DAGs need not itself be acyclic). Second, the library's absolute-magnitude threshold ($\omega = 0.3$) was applied inside each fit, and the historical union rule additionally retained positive coefficients only.

The revised protocol (Section 5.3) instead fits one DAGMA model per horizon on the offset-0 subsample, yielding a genuine single 28-edge DAG per horizon, and retains both coefficient signs after the library threshold. Its slice-0 fits reproduce the historical slice-0 graph supports exactly at PH=1, 2, and 4 (28/28 edges; 26/28 at PH=3), with shared-edge weights agreeing to within approximately 0.016; the revised per-horizon graphs are strict subsets of the historical unions. The historical tables below remain unchanged single-seed results of the original pipeline.

A.2 GCN Results

Table A1 shows the GCN results with physical graph, GSL, and cGSL.

Table A1: GCN performance with physical graph, GSL, and cGSL on both datasets (PH=1–4).

		SZ-taxi		Los-loop	
PH	Method	RMSE	MAE	RMSE	MAE
1	GCN	5.958	4.409	7.724	5.555
	GCN-GSL	4.886	3.161	7.527	5.065
	GCN-cGSL	**4.648**	**3.078**	**5.440**	**3.452**
2	GCN	5.983	4.437	7.940	5.692
	GCN-GSL	4.904	3.173	7.867	5.268
	GCN-cGSL	**4.672**	**3.051**	**5.806**	**3.613**
3	GCN	5.991	4.438	8.102	5.782
	GCN-GSL	4.958	3.223	8.073	5.453
	GCN-cGSL	**4.712**	**3.122**	**6.171**	**3.821**
4	GCN	6.002	4.450	8.285	5.876
	GCN-GSL	4.933	3.199	9.067	6.085
	GCN-cGSL	**4.726**	**3.107**	**6.745**	**4.097**

GCN-cGSL (symmetrized cyclic graph) consistently outperforms both standard GCN and GCN-GSL, with mean RMSE improvements over the physical graph of 21.6% on SZ-taxi and 24.7% on Los-loop (multi-horizon means over PH=1–4, computed from Table A1). This suggests that the purely spatial GCN benefits from bidirectional (cyclic) graph connections.

A.3 TGCN Results

Table A2 shows the TGCN results.

TGCN-GSL (acyclic DAG graph) outperforms both standard TGCN and TGCN-cGSL, achieving a mean 21.8% RMSE improvement over the physical graph on Los-loop (multi-horizon mean over PH=1–4, single seed, original pipeline; per-horizon maxima are higher, e.g., PH=1: 6.588 → 4.818, a 26.9% reduction). The acyclic structure aligns well with T-GCN's temporal modeling through the GRU component. Under the revised pipeline this comparison is reproduced at five seeds with a 21.7% mean improvement (25.5% at PH=1; Section 5.3), which independently validates the magnitude of the original claim even though the absolute RMSE values of the two pipelines are not directly comparable.

A.4 Key Observation

The asymmetry between GCN (cyclic preferred) and T-GCN (acyclic preferred) motivated the investigation into what the learned graph actually represents. The multi-lag formulation (main text) resolves this by making the temporal structure explicit rather than relying on post-hoc interpretation of a single ambiguous graph. The reproduction

Table A2: TGCN performance with physical graph, GSL, and cGSL on both datasets (PH=1–4).

PH	Method	SZ-taxi RMSE	SZ-taxi MAE	Los-loop RMSE	Los-loop MAE
1	TGCN	4.866	3.606	6.588	4.573
	TGCN-GSL	**4.214**	**2.797**	**4.818**	**2.980**
	TGCN-cGSL	4.821	3.537	6.550	4.553
2	TGCN	4.506	3.213	6.960	4.838
	TGCN-GSL	**4.239**	**2.801**	**5.400**	**3.410**
	TGCN-cGSL	4.534	3.276	6.915	4.830
3	TGCN	4.685	3.416	7.361	5.053
	TGCN-GSL	**4.344**	**3.062**	**5.846**	**3.467**
	TGCN-cGSL	4.630	3.334	7.331	5.054
4	TGCN	4.934	3.638	7.568	5.166
	TGCN-GSL	**4.366**	**2.885**	**6.257**	**3.805**
	TGCN-cGSL	4.774	3.487	7.539	5.172

analysis additionally showed that this original single-graph construction was contemporaneous (Section 5.3): the DAGs above encode present-observation functional dependencies, and the temporal interpretation is supplied by the explicitly lagged multi-lag construction of Section 3.3.

Appendix B Bibliometric Analysis Methodology

To justify the focus on graph-based approaches in traffic forecasting, we conducted a systematic bibliometric analysis.

B.1 Data Collection

A broad search in Scopus using "traffic forecast*" retrieved approximately 60,000 documents. Filtering for peer-reviewed English articles in Engineering or Computer Science yielded 784 articles (2000–2025).

B.2 Keyword Processing

The 6,343 unique keywords were reduced to 217 (frequency ≥ 10). Semantic clustering using Sentence-BERT embeddings grouped these into 66 meaningful concept clusters. Similar terms (e.g., "graph neural network", "GNN") were unified.

B.3 Key Findings

Figure B1 reveals that graph-based methods have become the dominant recent methodological focus. However, nearly all rely on predefined graph structures—directly motivating our Graph Structure Learning approach.

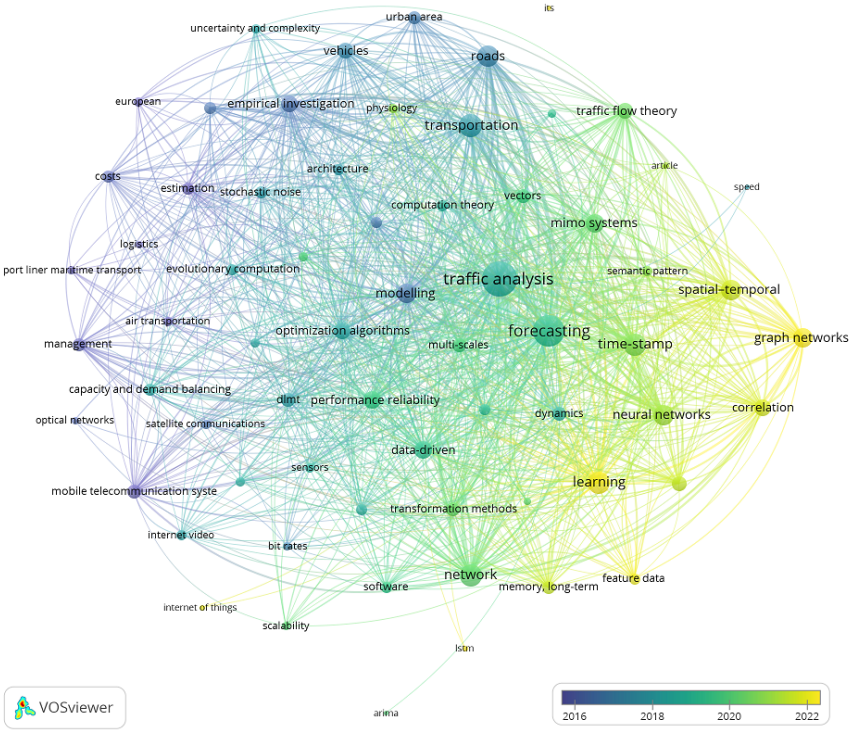
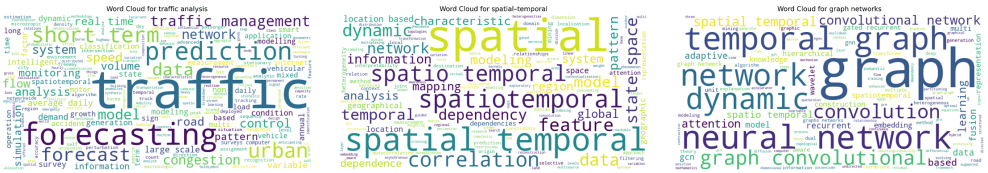


Fig. B1: Clustered keyword co-occurrence network from 784 articles, colored by average publication year. Graph-based methods (right nodes) are the most recent focus.



(a) Traffic Analysis (b) Spatio-Temporal (c) Graph Networks

Fig. B2: Word clouds from three key bibliometric clusters.

Appendix C Additional Diagnostics

C.1 Graph Density Statistics

Table C3 compares graph densities across methods.

C.2 DAGMA Threshold Sensitivity

Figure C3 reports the sensitivity of Los-loop results to the DAGMA threshold τ (equivalently, to the learned graph's sparsity). RMSE decreases monotonically as sparser graphs are selected, and T-GCN-MultiGSL-Mix attains its best RMSE near the operating point of 30 edges used throughout the main text.

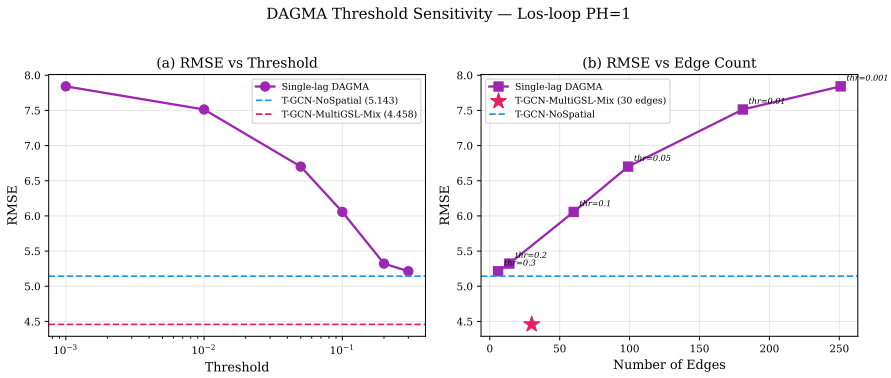


Fig. C3: DAGMA threshold sensitivity (Los-loop, PH=1, seed=42). (a) RMSE decreases monotonically with higher thresholds. (b) RMSE vs. edge count: T-GCN-MultiGSL-Mix (star) achieves the best performance with 30 edges.

C.3 Training Convergence (Revision Experiments)

Figure C4 shows the training loss curves for the three main methods on Los-loop and SZ-Taxi. All methods converge within 50 epochs, with T-GCN-MultiGSL-Mix achieving the lowest final training loss on Los-loop; on SZ-Taxi the curves are close, consistent with the marginal performance difference.

C.4 Parameter-Matched Control

Figure C5 visualizes the parameter-matched control of the main text (Table 5).

Table C3: Graph density comparison (Los-loop, $N = 207$).

Graph	Edges	Density
Physical	2833	0.0660
T-GCN-NoSpatial (identity)	207	0.0048
T-GCN-MultiGSL (τ=0.1)	30	0.0007
T-GCN-MultiGSL-Mix (τ=0.1)	30	0.0007

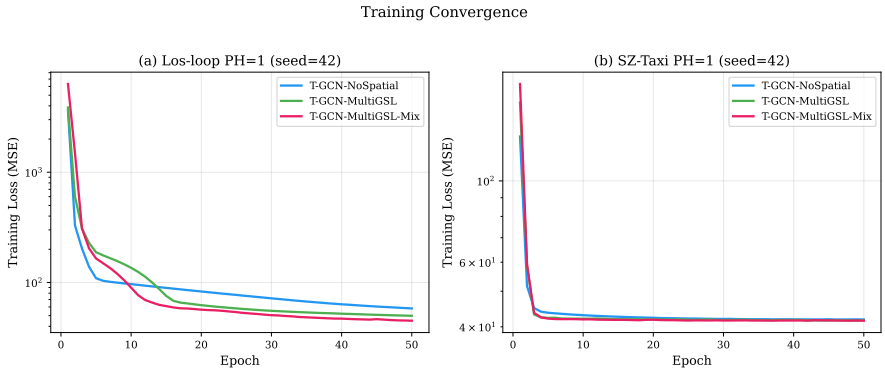


Fig. C4: Training convergence (PH=1, seed=42). (a) Los-loop: T-GCN-MultiGSL-Mix converges to the lowest training loss. (b) SZ-Taxi: all methods show similar convergence, consistent with the marginal performance difference.

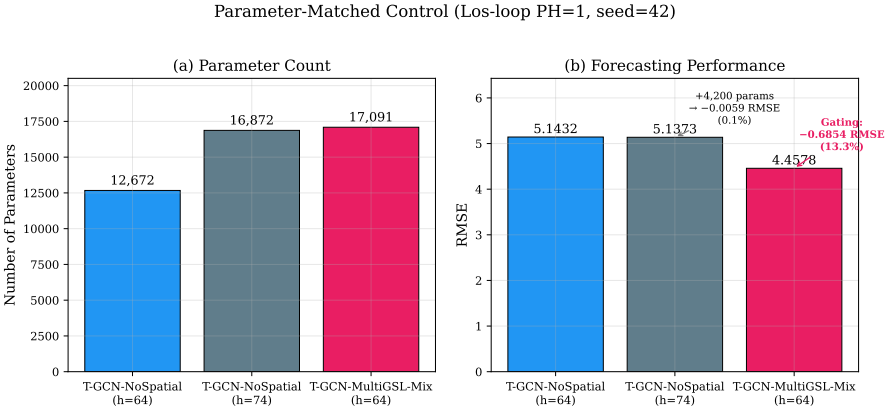


Fig. C5: Parameter-matched control (Los-loop, PH=1, seed=42). (a) Parameter counts are comparable. (b) Adding 35% more parameters to T-GCN-NoSpatial improves RMSE by only 0.1%, while the gating architecture provides 13.3% improvement.

C.5 DAGMA Computational Cost

Multi-lag DAGMA runtime on standard hardware (RTX 3090):

- SZ-Taxi (624×624): approximately 100 minutes per prediction horizon
- Los-loop (828×828): approximately 240 minutes per prediction horizon

Total computation for the 8 primary DAGMA runs (2 datasets $\times$ 4 PHs): approximately 24 hours; the additional 15-minute-resolution DAGMA run (Section 5.6) took approximately 4 hours.

For reference, the contemporaneous single-graph fits of Section 5.3 (207 variables, $\lambda_1 = 0.02$, warm-up 30,000 / max 60,000 iterations) took approximately 15–17 minutes per horizon on 4 CPU cores, and archived 156-variable SZ-Taxi audit fits took approximately 52–76 minutes each. A sliding-window time-varying extension would multiply these per-fit costs by the number of windows.

References

[1] Akhtar, M., Moridpour, S.: A review of traffic congestion prediction using artificial intelligence. Journal of Advanced Transportation **2021** (2021)

[2] Kipf, T.N., Welling, M.: Semi-supervised classification with graph convolutional networks. In: ICLR (2017)

[3] Zhao, L., Song, Y., Zhang, C., Liu, Y., Wang, P., Lin, T., Deng, M., Li, H.: T-GCN: A temporal graph convolutional network for traffic prediction. IEEE Trans. on Intell. Transportation Systems **21**(9), 3848–3858 (2020)

[4] Chen, Y., Wu, L.: In: Wu, L., Cui, P., Pei, J., Zhao, L. (eds.) Graph Neural Networks: Graph Structure Learning, pp. 297–321. Springer, Singapore (2022). https://doi.org/10.1007/978-981-16-6054-2_14 . https://doi.org/10.1007/978-981-16-6054-2_14

[5] Zheng, X., Aragam, B., Ravikumar, P., Xing, E.P.: Dags with no tears: Continuous optimization for structure learning. In: NeurIPS (2018)

[6] Bello, K., Aragam, B., Ravikumar, P.: Dagma: learning dags via m-matrices and a log-determinant acyclicity characterization. In: Proceedings of the 36th International Conference on Neural Information Processing Systems. NIPS '22. Curran Associates Inc., Red Hook, NY, USA (2024)

[7] Amintoosi, M.: Urban traffic prediction with learning based graph convolutional networks. In: Proceedings of the 55^{th} Annual Iranian Mathematics Conference, Ferdowsi University of Mashhad, pp. 145–148 (2024). (In Persian). https://github.com/mamintoosi/TGCN-GSL-TF